Tab 1

BUDGET TRACKER

Group 2 of 8 - Sampaguita

Members:

Alipalo, Jaden Skye

Aurin, Kurt Yuji

Casipit, Gianna Solemn

De Mesa, Brooklyn

Manlangit, Gian Peter

.

Rationale

Money management is an essential aspect of life, and many students enter the world of managing their own finances without any organized instructions or knowledge. In Philippine science high school, students receive weekly allowances that are fixed to cater to the basic needs and cost of the school projects and meals. Nonetheless, many students have difficulties with budgeting, as they tend to spend money on the spur of the moment, and do not save regularly.

This scenario indicates a disconnect between a financial responsibility and the practical financial management skills. Although students can learn about the need to save and budget in theory, they have no straightforward and ready-to-use system to put these concepts in the real world. Consequently, they end up having poor spending habits that could extend even after their education.

The formation of Budget Tracker fills this gap by offering a well-organized but easy to use tool which could help students to actively control their allowances. The system turns abstract financial notions into actionable steps by enabling the user to plan and set allocations, keep track of expenditures, balance, and trends in expenditures. It helps students to be more conscious with their expenditure choices and to consider necessities over desires.

Moreover, the simplicity and portability of the system will guarantee its compatibility with the lifestyle of students, who are in need of fast and portable solutions. This Budget Tracker is designed to meet specific needs and behavior of limited-resource students, unlike complex financial applications that are made to meet the needs of adults.

Finally, the project is also not solely about the money tracking but also financial discipline, responsibility, and awareness among youths. With the early introduction of these habits, the system will assist students in having better financial habits that can be useful in the long run.

Project Overview and Relevance

Our project, Budget Tracker is an easy to use software system in which it helps Pisay scholars to better organize and utilize their weekly allowances. With this application, users will have the chance to input their allowances for the week, allocate them to certain categories, namely cafeteria, co-op, school projects, and savings, while recording their expenses every day. This application will enable users to have constant updates of their balances per category as well as a detailed report about all their transactions in one convenient interface.

Moreover, users will receive helpful summaries and other basic reports that would show their expenditures and savings. Low balance and percentage summaries would provide assistance in managing the money more efficiently and avoiding overspending. Overall, this system is very convenient and user-friendly thus prompting frequent usage among busy students.

The project is very relevant as it solves problems that most students face nowadays, specifically in managing their small weekly allowances. From multiple perspectives, it occurs that many Pisay scholars have problems with overspending, savings, and keeping track of their finances because there is no systematic approach to doing that. In order to solve this problem, the Budget Tracker application was created as a convenient and useful tool.

Thus, through this system, scholars can become financially literate and gain experience in important financial activities such as budgeting, planning purchases, and making necessary expenditures. Besides, with the application, the process of allocating funds will be easier and faster thus making scholars less dependent on their family members, more responsible, and disciplined.

Through the use of this application, students are introduced to fundamental concepts of financial management such as expense tracking, budgeting awareness, and spending analysis. Although the system is designed to be simple and accessible, it serves as a practical tool for promoting responsible financial behavior and improving students' ability to manage their personal finances effectively.

Objectives

The primary objective of the program, Budget Tracker, is to develop a simple and functional Budget Tracker that assists Philippine Science High School (Pisay) scholars in managing their weekly allowances more effectively.

With this program, the programmers specifically aim to:

1. To help students improve their budgeting practices by providing a structured system for recording and monitoring their daily expenses over a weekly period.
2. To enhance students' awareness of their spending habits by allowing them to regularly review their transactions and observe patterns in their expenditures.

3. To encourage responsible financial behavior, such as prioritizing needs over wants and maintaining control over limited financial resources, through consistent use of the system during the testing period.
4. To support students in making more informed financial decisions by giving them clear and updated information about their remaining balances across different spending categories.

5. To determine the effectiveness of the Budget Tracker in promoting better financial management among students based on its ability to assist users in organizing, tracking, and reviewing their expenses throughout one budgeting cycle.
6. To provide an accessible and easy-to-use tool that fits the daily routines of students, thereby increasing the likelihood of consistent usage and long-term adoption.

These objectives aim to evaluate not only the functionality of the system but also its potential contribution to improving students' financial awareness and discipline

.

Methodology

This study adopts a software development approach in designing and implementing a Budget Tracker system using Python as the primary programming language and JSON as the data storage format. The system is developed as a command-line interface (CLI) application to ensure simplicity, accessibility, and ease of use for students managing their weekly allowances.

Python is used to handle the core functionality of the system, including user authentication, expense recording, category management, and data processing. The program is

structured using functions to organize different features such as account creation, login validation, transaction logging, and financial analysis. These implementations are consistent with standard Python programming practices for building data-driven applications, which emphasize the use of loops, conditionals, and modular functions for efficient program flow

For data persistence, the system utilizes JSON (JavaScript Object Notation) as a lightweight storage solution. User data, including accounts, categories, and transactions, are stored in a JSON file and managed using Python's file handling capabilities. Functions such as json.load() and json.dump() allow the program to retrieve and save data efficiently, ensuring that user inputs are preserved even after the program is closed (Acsany, 2025). This approach is commonly used in beginner to intermediate-level applications due to its simplicity and portability

The system implements basic CRUD (Create, Read, Update, Delete) operations to manage financial data. Create operations are applied when users register accounts or add new expenses. Read operations are used when displaying transaction logs and balance summaries. Update operations occur when category balances are adjusted after expenses or when data is reset for a new budgeting cycle. Delete operations are implemented through the removal of incorrect or unwanted expense entries. Similar Python-based expense tracker systems in academic projects also utilize CRUD operations combined with dictionaries and lists for efficient data handling and analysis (Bhavani et al., 2025; *Client Challenge*, n.d.).

For data analysis, the system processes stored transactions to generate simple financial summaries, such as total spending per category and identification of the highest spending category. These analyses are performed using basic Python data structures such as lists and

dictionaries, which are commonly used in educational financial applications due to their flexibility and ease of use (Zhu, 2024). Studies also show that Python-based financial tools can improve users' financial awareness and decision-making by promoting structured analysis of spending behavior (Zhu, 2023b)

User Login and Account Creation System

- Uses input prompts to allow users to either register or log in.
- Account creation stores username and password as key-value pairs in a JSON dictionary.
- Login system checks entered credentials against stored JSON data using if conditions.
- Uses loops to repeatedly ask for input until valid credentials are entered.
- Prevents unauthorized access by only allowing matching username-password pairs.

Weekly Allowance Input

- Accepts numeric input from the user for their weekly budget.
- Stores the allowance in a variable and saves it to JSON for persistence.
- Acts as the main budget source for all spending calculations.
- Uses simple arithmetic to initialize available balance.

Budget Allocation per Category

- Uses a dictionary to store categories (e.g., cafeteria, co-op, projects, savings).
- Each category is assigned a portion of the total allowance.
- Values are updated using assignment operations (category[key] = value)
- Allows structured tracking of where money is allocated.

• Expense Entry System

- Prompts user to input expense amount and select a category.
- Uses conditional statements to validate category selection.
- Deducts expense from both total balance and specific category value.
- Stores expense data as a dictionary inside a list of transactions.
- Ensures real-time updating of budget after every entry.

• Transaction Logging

- Stores all user expenses in a transaction list.
- Each transaction is saved as a dictionary (amount, category, etc.).
- Uses .append() to continuously add new transactions.
- Data is written to JSON using json.dump() for permanent storage.
- Allows full history tracking of financial activity.

• Dashboard-Based Menu Navigation

- Main interface is a CLI menu system using loops.
- Displays numbered options for different functions (add expense, view balance, etc.).
- Uses if-elif statements to execute selected features.
- Runs continuously until user chooses to exit.
- Acts as the central controller of the program flow.

Additional Features

Savings Tracking Category

- Treated as a separate budget category inside the dictionary.
- Funds allocated here are stored separately from spending categories.
- Not directly reduced unless explicitly updated by the user.
- Helps simulate saving behavior in budgeting.

• Customizable Categories (Add/Remove)

- Uses dictionary operations to add or delete categories dynamically.
- input() allows users to define new category names.
- del or removal functions are used to delete categories.
- Includes validation to avoid duplicates or invalid inputs.

• Spending Analysis per Category

- Iterates through transaction list using loops.
- Groups expenses by category using dictionary accumulation.
- Computes total spending per category using addition logic.
- Helps identify which categories have the highest spending.

• Smart Summaries and Full Reports

- Uses stored data to compute total balance, expenses, and remaining funds.
- Aggregates values using loops and arithmetic operations.
- Displays formatted output summaries in the CLI.
- Provides users with a full financial overview.

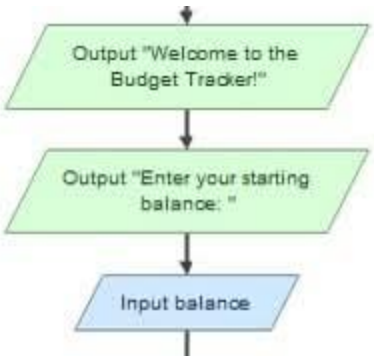
• Low Balance Alert System

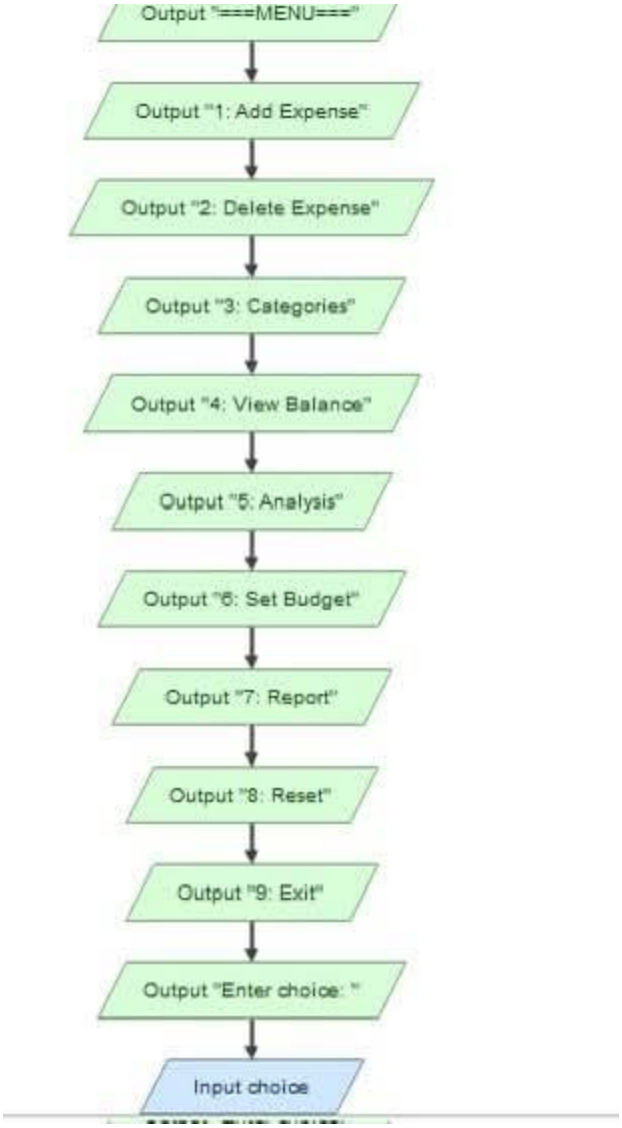
- Checks remaining balance after every transaction.
- Uses if balance <= threshold condition to trigger alert.
- Displays warning message in the CLI automatically.
- Integrated inside expense or update function for real-time monitoring.

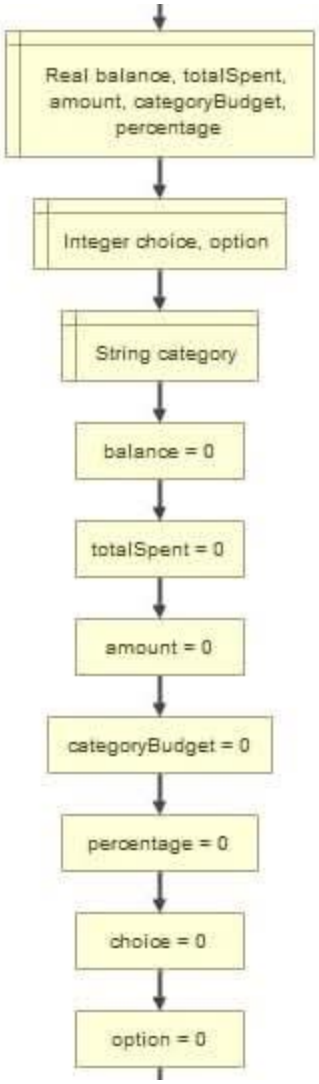
• Weekly Reset Function

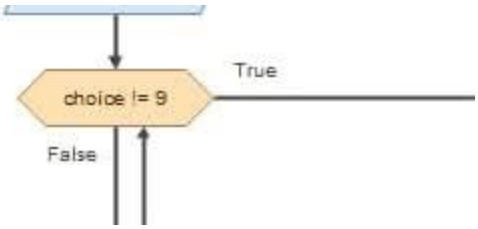
- Resets budget values at the end of each cycle.
- Clears or archives transaction list depending on design.
- Reinitializes category values back to default.
- Updates JSON file using json.dump() save reset state.
- Uses a function triggered manually or through menu option.

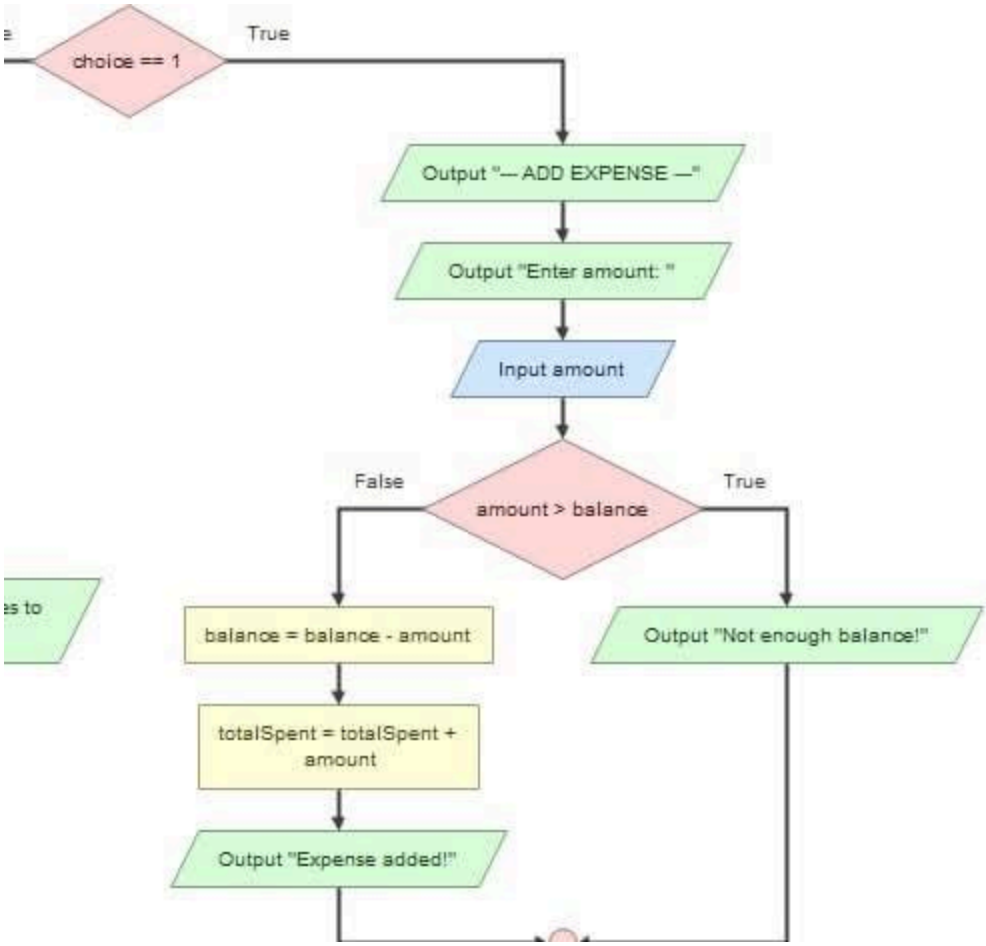
Flowchart

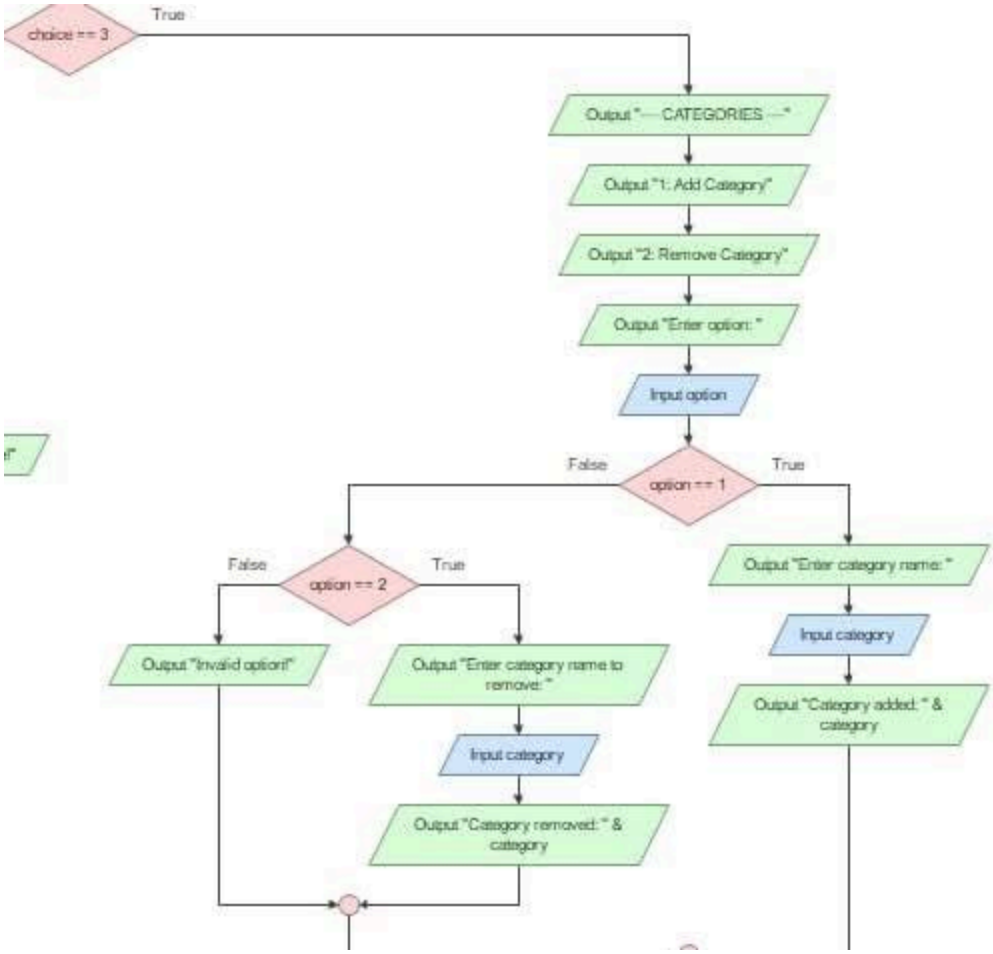

Output "Welcome to the Budget Tracker!"
Output "Enter your starting balance: "
Input balance

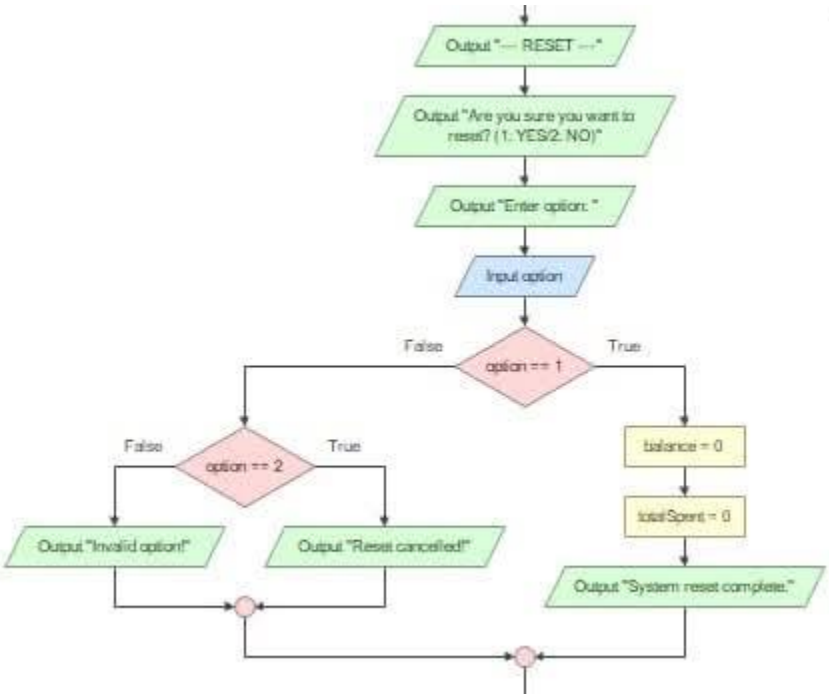

Output "===MENU===
Output "1: Add Expense"
Output "2: Delete Expense"
Output "3: Categories"
Output "4: View Balance"
Output "5: Analysis"
Output "6: Set Budget"
Output "7: Report"
Output "8: Reset"
Output "9: Exit"
Output "Enter choice: "
Input choice

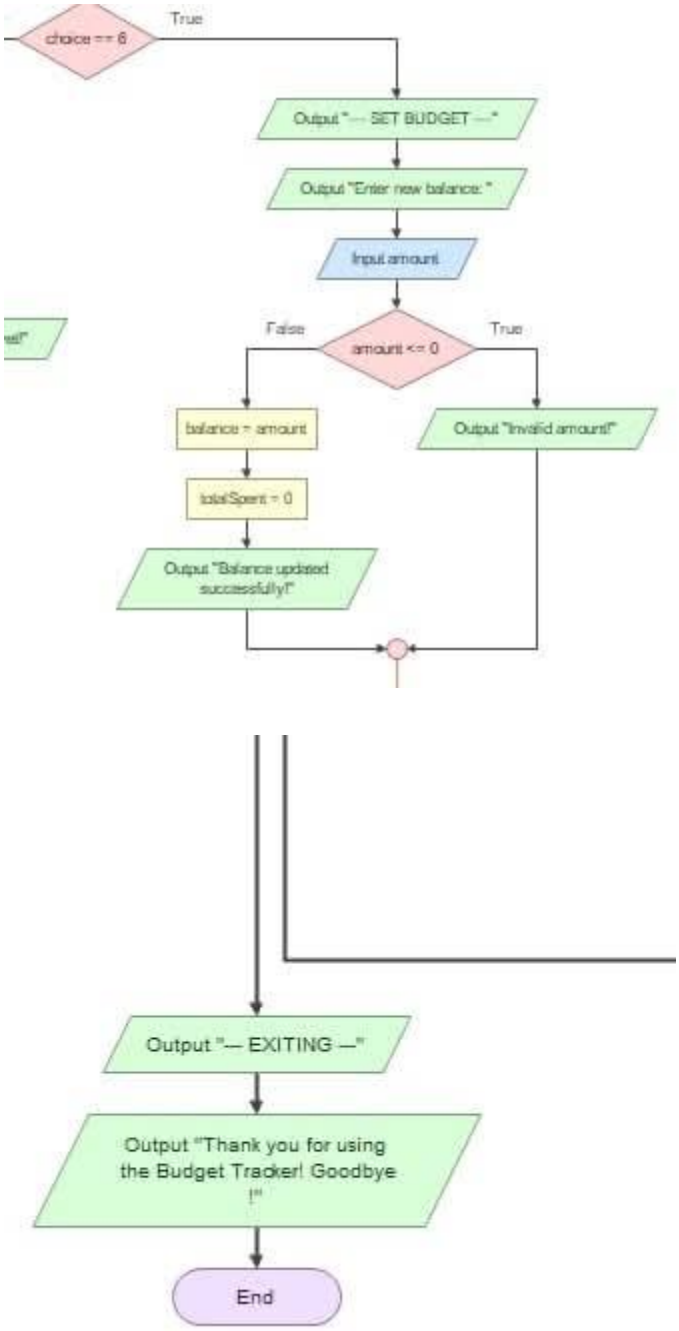

Real balance, totalSpent, amount, categoryBudget, percentage
Integer choice, option
String category
balance = 0
totalSpent = 0
amount = 0
categoryBudget = 0
percentage = 0
choice = 0
option = 0

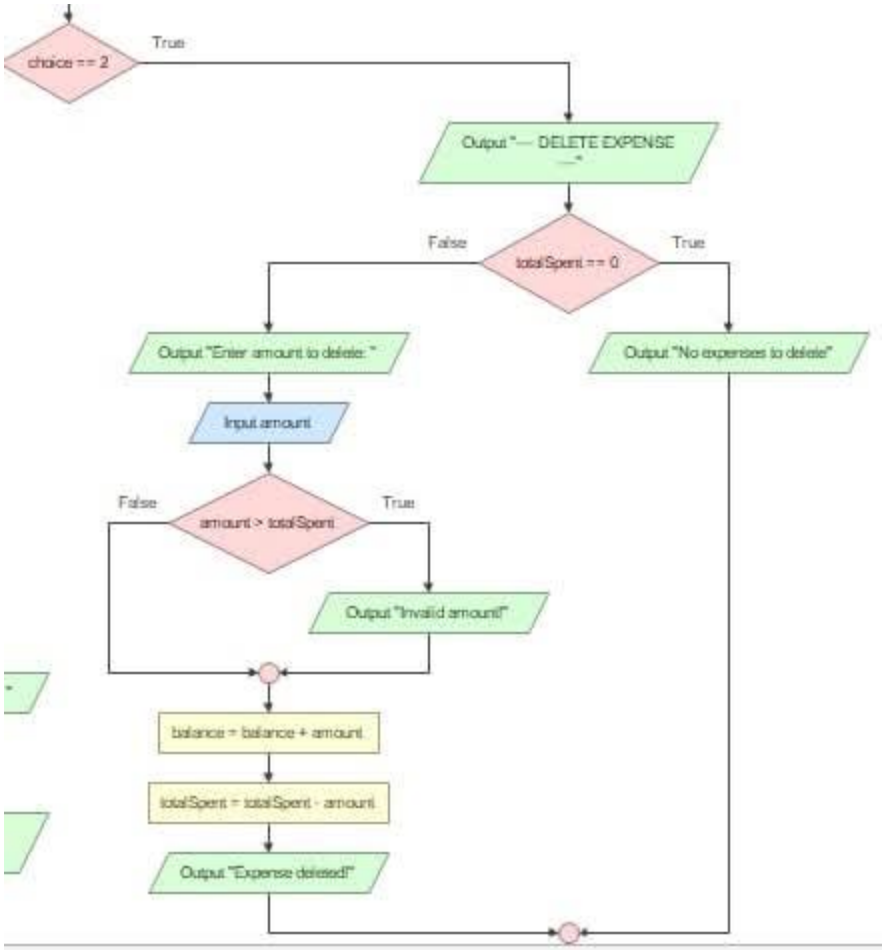

choice != 9
True
False

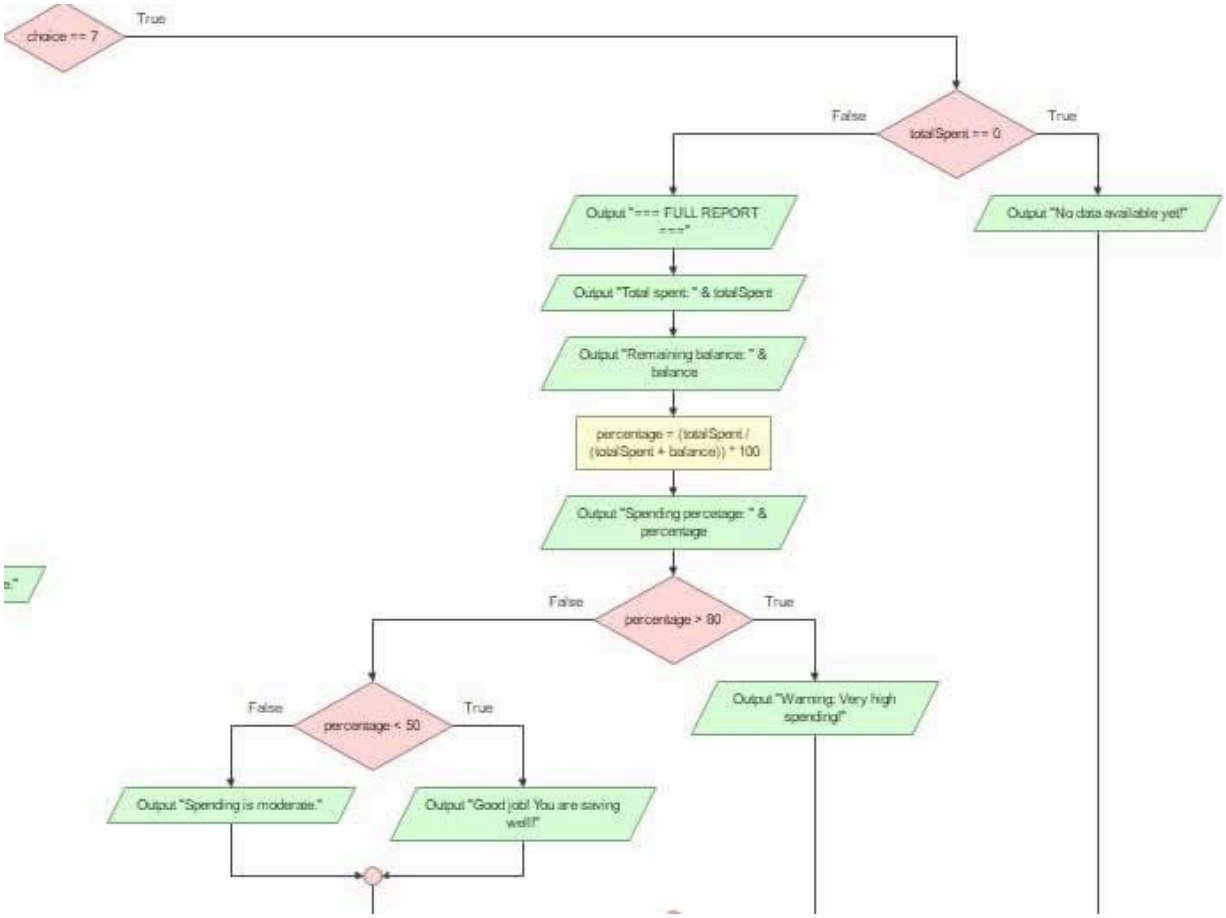

choice == 1
True
Output "— ADD EXPENSE —"
Output "Enter amount: "
Input amount
False
amount > balance
True
balance = balance - amount
Output "Not enough balance!"
totalSpent = totalSpent + amount
Output "Expense added!"

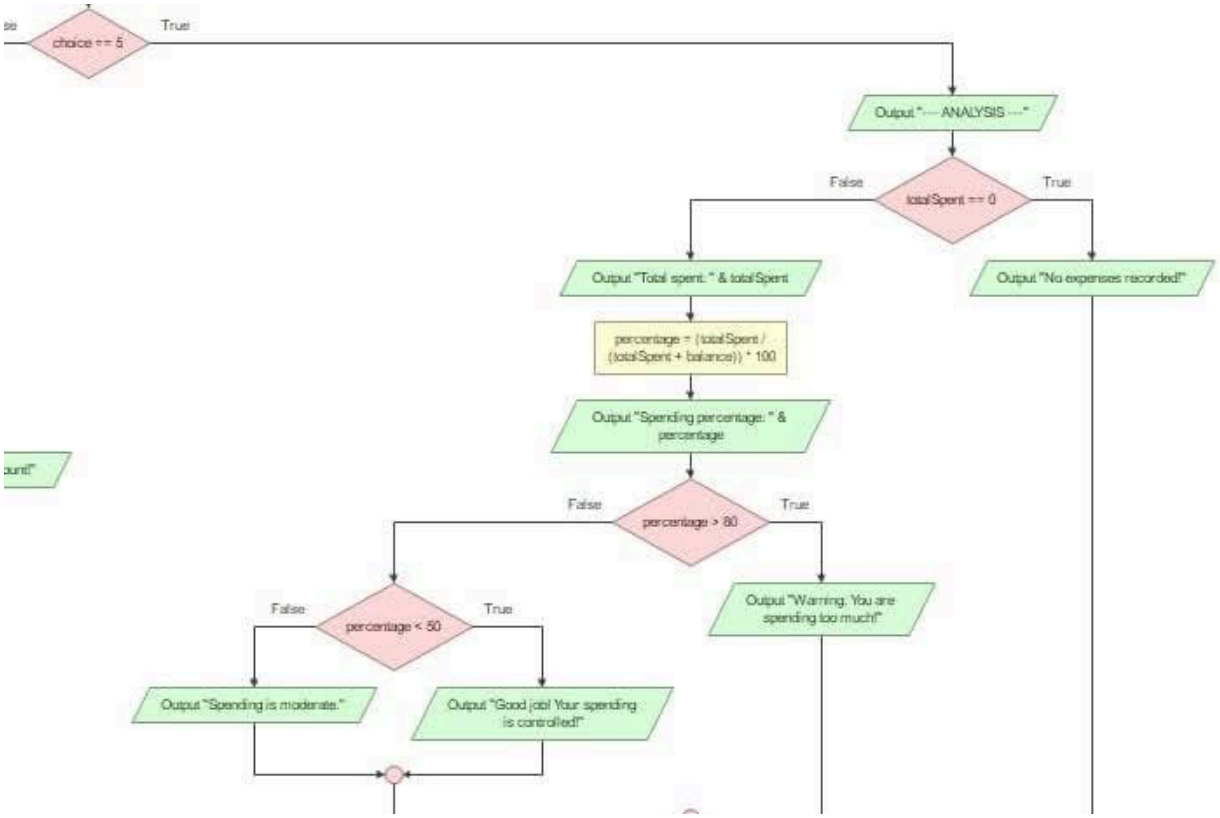

choice == 3
True
Output "— CATEGORIES —"
Output "1: Add Category"
Output "2: Remove Category"
Output "Enter option: "
Input option
False
option == 1
True
False
option == 2
True
Output "Enter category name: "
Input category
Output "Invalid option!"
Output "Enter category name to remove: "
Output "Category added: " & category
Input category
Output "Category removed: " & category

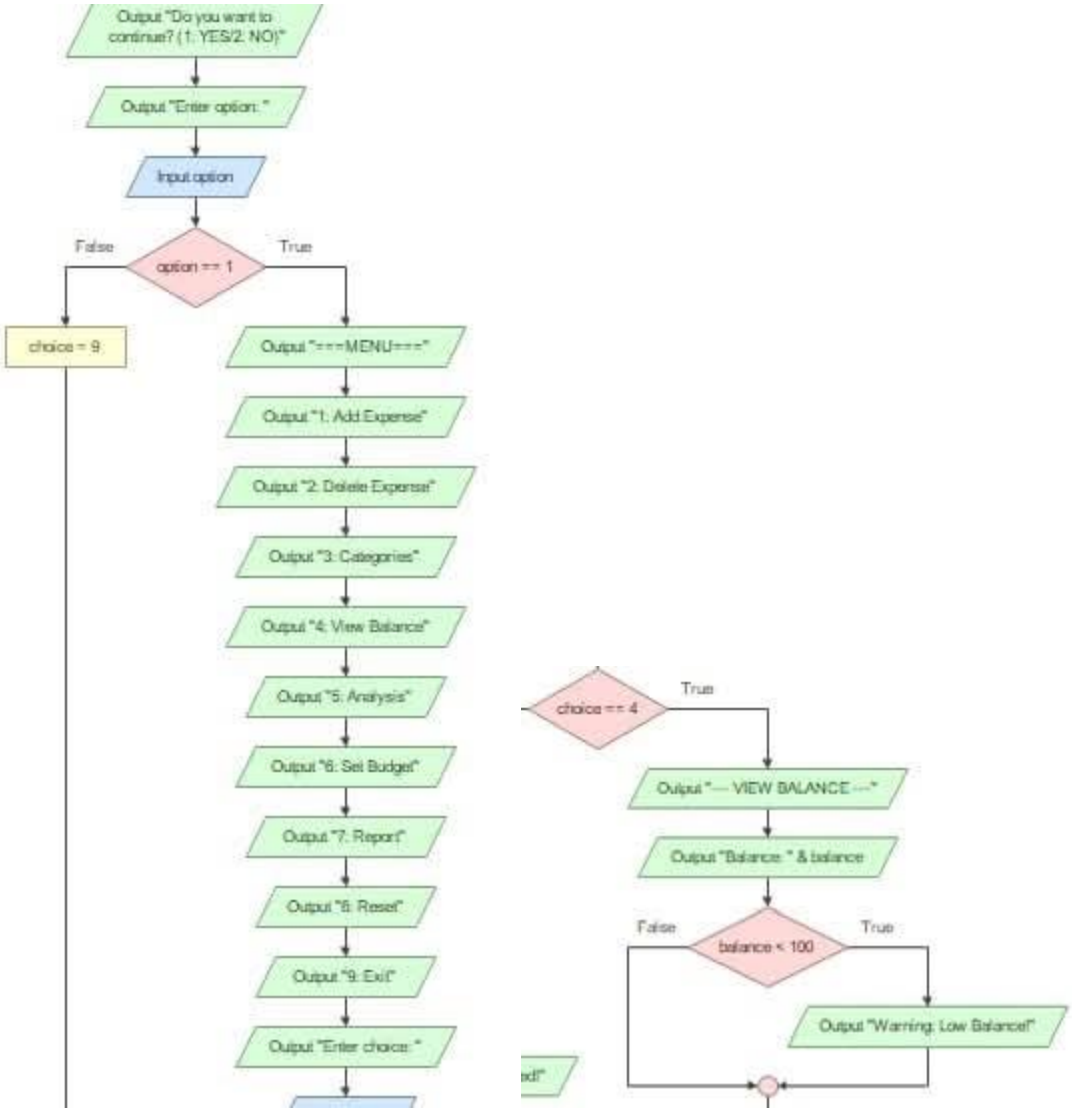

Output "— RESET —"
Output "Are you sure you want to reset? (1: YES/2: NO)"
Output "Enter option: "
Input option
False
option == 1
True
False
option == 2
True
balance = 0
totalSpent = 0
Output "Invalid option!"
Output "Reset cancelled!"
Output "System reset complete."


choice == 6
True
Output "— SET BUDGET —"
Output "Enter new balance: "
Input amount
False
amount <= 0
True
balance = amount
Output "Invalid amount!"
totalSpent = 0
Output "Balance updated successfully!"
Output "— EXITING —"
Output "Thank you for using the Budget Tracker! Goodbye !"
End


choice == 2
True
Output "— DELETE EXPENSE —"
False
totalSpent == 0
True
Output "Enter amount to delete: "
Output "No expenses to delete"
Input amount
False
amount > totalSpent
True
Output "Invalid amount!"
balance = balance + amount
totalSpent = totalSpent - amount
Output "Expense deleted!"


choice == 7
True
totalSpent == 0
False
True
Output "No data available yet!"
Output "=== FULL REPORT ===="
Output "Total spent: " & totalSpent
Output "Remaining balance: " & balance
percentage = (totalSpent / (totalSpent + balance)) * 100
Output "Spending percetage: " & percentage
percentage > 80
False
True
Output "Warning: Very high spending!"
percentage < 50
False
True
Output "Spending is moderate."
Output "Good job! You are saving well!"


choice == 5
True
Output "--- ANALYSIS ---"
False
totalSpent == 0
True
Output "Total spent: " & totalSpent
Output "No expenses recorded!"
percentage = (totalSpent / (totalSpent + balance)) * 100
Output "Spending percentage: " & percentage
False
percentage > 80
True
Output "Warning. You are spending too much!"
False
percentage < 50
True
Output "Spending is moderate."
Output "Good job! Your spending is controlled!"


Output "Do you want to continue? (1: YES/2: NO)"
Output "Enter option: "
Input option
False
option == 1
True
choice = 9
Output "===MENU===="
Output "1: Add Expense"
Output "2: Delete Expense"
Output "3: Categories"
Output "4: View Balance"
Output "5: Analysis"
Output "6: Set Budget"
Output "7: Report"
Output "8: Reset"
Output "9: Exit"
Output "Enter choice: "
choice == 4
True
Output "— VIEW BALANCE —"
Output "Balance: " & balance
False
balance < 100
True
Output "Warning: Low Balance!"

Program

Team Roles

	Alipalo	Aurin	Casipit	De Mesa	Manlangit
1st Quarter	Wrote the feasibility & scope	Wrote the problem-statement	Wrote the problem-identification and proposed solution, formatted the file	Wrote the problem decomposition	
2nd Quarter	Created logic/code for flowchart		Formatting of file, revising and adding to problem decomposition, created the logic plan/flowchart	Wrote the problem decomposition	
3rd Quarter	Contributed to readme.md	Contributed to the code (User log-in system)	Contributed to the code (Main Menu/Dashboard)	Wrote readme.md file, contributed to the code (Spending analysis, delete expenses)	Wrote the readme.md file, contributed to the code (Log in system)
4th Quarter	Flowchart and screenshots of flowchart	Rationale, contributed to code (Preventing overspending alert)	Revisions and slight add-ons to Rationale and Project Overview, Formatting of file, Methodology, Objectives, contributed to code	Project Overview and Relevance, contributed to code (Budget allocation)	Screenshots of program

			(percentage/smart summary, low balance alert, full report), Screenshots of code		

References

Acsany, P. (2025, August 20). *Working with JSON data in Python*. https://realpython.com/python-json/

Bhavani, V. T., Chinthana, S., Aishwarya, K., Shweta, & Nirmala, N. (2025). *Implementation of an expense tracker using Python*. International Journal for Multidisciplinary Research, 7(4), 1–9. https://www.ijfmr.com/papers/2025/4/50442.pdf

Zhu, A. Y. F. (n.d.). Optimizing financial decision-making for emerging adults: A compact Python-based personalized financial projection approach. *Technology in Society*, *77*, 102599. https://doi.org/10.1016/j.techsoc.2024.102599

Zhu, A. Y. F. (2023). Upgrading financial education by adding Python‐based personalized financial projection: A randomized control trial. *British Journal of Educational Technology*, *55*(2), 731–750. https://doi.org/10.1111/bjet.13401

Tab 2

Budget Tracker

The **Budget Tracker** is a simple console-based Python program designed to help students manage their weekly allowance. It allows users to track expenses, allocate budgets by category, and monitor remaining balances.

The system also promotes better financial habits by helping users understand their spending patterns and prioritize savings.

—

Objectives

- Help students track daily expenses
- Allow users to allocate their allowance into different categories
- Monitor remaining budget per category and overall balance
- Encourage better saving and budgeting habits
- Provide simple spending summaries and reports

—

Features

Core Features

- User login and account creation system

- Weekly allowance input

- Budget allocation per category

- Expense entry system

- Transaction logging

- Dashboard-based menu navigation

Additional Features

- Savings tracking category

- Customizable categories (add/remove)

- Spending analysis per category

- Smart summaries and full reports

- Low balance alert system

- Weekly reset function

Installation

1. Make sure Python 3 is installed

2. Download or copy the project files

3. Open Command Prompt / Terminal

4. Navigate to the project folder

5. (Optional) Install dependencies:

   ```bash

   pip install -r requirements.txt

6. Run the program by typing python budget_tracker.py

7. The program will start and show the dashboard

—

## Usage

**Dashboard Menu:**

```
def dashboard():

    while True:

        print("\n===== DASHBOARD =====")
```
```

```
    print("1 - Add Expense")
    print("2 - View Balance")
    print("3 - Set Up Budget")
    print("4 - Savings Tracker")
    print("5 - Balance Summary")
    print("6 - Spending Analysis")
    print("7 - Transaction Log")
    print("8 - Delete Expense")
    print("9 - Add Category")
    print("10 - Remove Category")
    print("11 - Smart Summary")
    print("12 - Full Report")
    print("13 - Reset Week")
    print("14 - Logout")
```

—

Sample Output

===== BUDGET TRACKER LOG IN =====

1 - Login

2 - Create Account

Select Option: 1

Enter username: kiyozn

Enter password: 123

Login succesful!

=====

===== DASHBOARD =====

1 Add Expense

2 View Balance

3 Set Up Budget

4 Savings

5 Summary

6 Analysis

7 Log

8 Delete Expense

9 Add Category

10 Remove Category

11 Smart Summary

12 Full Report

13 Reset

14 Logout

Choice: 1

—

Data Storage

The program uses local file:

data.json

It stores:

- User accounts
- Categories
- Transactions

—

Contributing

- Fork or copy the project

- Make improvements or fixes
- Test your changes
- Submit with explanation

Keep the program simple and beginner-friendly.

License

This project is for educational purposes only.

Students and teachers may use and modify it for learning.
Commercial use is not allowed without permission.

© 2026 Budget Tracker Developers.